

Tutorial - 7

(Time complexity analysis)

Q.1) Which of the following is not $O(n^2)$?

(A) $(15^{10}) * n + 12099$

(B) $n^{1.98}$

(C) $n^3 / (\text{sqrt}(n))$

(D) $(2^{20}) * n$

Solution: (C)

Explanation: The order of growth of option C is $n^{2.5}$ which is higher than n^2 .

Q.2) Find the time complexity of the below code snippet.

```
for (i = 0; i < n; i++)
{
    for (j = 0; j < i; j++)
    {
        Stmt;
    }
}
```

Solution: Assuming 'Stmt' takes a constant time, we need to check how many times the statement runs in total by finding the number of times it runs for different values of i and j.

i	j	Statement
0	0	X → 0 times
1	0	1 → 1 times
	1	X
2	0	1 → 2 times

	1	1	
	2	X	
3	0	1	→ 3 times
	1	1	
	2	2	
	3	X	
n-1			→ n-1 times

Total: $1 + 2 + 3 + \dots + n-1 = \frac{n(n-1)}{2} = O(n^2)$

Q.3) Find the time complexity of the below code snippet.

```

p=0;
for (i=1; p<=n; i++)
{
    p= p+i;
}

```

Solution: Here it is clear that the statement $p=p+i$ is a simple addition statement and hence takes a constant time. So we just need to find out the total number of times this statement runs, as a function of n by checking how p changes as i changes.

i	p
1	$0 + 1 = 1$
2	$1 + 2 = 3$
3	$1 + 2 + 3 = 6$
.	
.	
.	
k	$1 + 2 + 3 + \dots + k-1 + k$

Assuming that after k iterations, $p > n$ and the loop will stop

So the loop stops when the following conditions are met:

$$p = k(k+1)/2 > n$$

$$\Rightarrow k^2 > n$$

$$\Rightarrow k = \sqrt{n} = O(\sqrt{n})$$

Q.4) Find the time complexity of the below code snippet.

```
void fun(int n, int arr[])
{
    int i = 0, j = 0;
    for(; i < n; ++i)
        while(j < n && arr[i] < arr[j])
            j++;
}
```

- (A) $O(n)$
- (B) $O(n^2)$
- (C) $O(n \log n)$
- (D) $O(n(\log n)^2)$

Solution: (A)

Explanation: In the first look, the time complexity seems to be $O(n^2)$ due to two loops. But, please note that the variable **j** is not initialized for each value of variable **i**. So, the inner loop runs at most **n** times.

Q.5) In a competition, four different functions are observed. All the functions use a single for loop and within the for loop, the same set of statements are executed. Consider the following for loops:

- A) for(i = 0; i < n; i++)
- B) for(i = 0; i < n; i += 2)
- C) for(i = 1; i < n; i *= 2)
- D) for(i = n; i > -1; i /= 2)

If **n** is the size of input(positive), which function is most efficient(if the task to be performed is not an issue)?

Solution: (C)

Explanation: The time complexity of the first for loop is $O(n)$.

The time complexity of the second for loop is $O(n/2)$, equivalent to $O(n)$ in asymptotic analysis.

The time complexity of the third for loop is $O(\log n)$.

The fourth for loop doesn't terminate.

Q.6) Find the time complexity of the below code snippet.

```
p=0;
for (i=1; i<n; i=i*2)
{
    p++;
}
for (j=1; j<p; j=j*2)
{
    Stmt;
}
```

Solution:

i	Statement (p)
---	---------------

1	1
---	---

1*2	2
-----	---

2*2	3
-----	---

2*2*2	4
-------	---

;

;

 $2^k > n$ $k * \log_2 2 = \log_2 n$ $p = k = \log_2 n$

j	statement
---	-----------

1	1
---	---

1*2	1
-----	---

2*2	1
-----	---

;

;

;

 $2^k = p$ $2^k = p = \log_2 n$ $k * \log_2 2 = \log_2 \log_2 n$ $O(\log_2 \log_2 n)$

Q.7) Find the time complexity of the below code snippet.

```

for (i=0; i<n; i++)
{
    for (j=1; j<n; j=j*2)
    {
        Stmt;
    }
}

```

Solution:

i	j	Statement
---	---	-----------

0	1	1
---	---	---

	1*2	1
--	-----	---

	2*2	1
--	-----	---

		;
--	--	---

		;
--	--	---

	$2^k = n$	
--	-----------	--

	$k = \log(n)$	
--	---------------	--

1	1	1
---	---	---

	1*2	1
--	-----	---

		;
--	--	---

		;
--	--	---

	$2^k = n$	
--	-----------	--

	$k = \log(n)$	
--	---------------	--

n-1	1	1
-----	---	---

	1*2	1
--	-----	---

		;
--	--	---

		;
--	--	---

	$2^k = n$	
--	-----------	--

	$k = \log(n)$	
--	---------------	--

$O(n \log n)$

Q.8) What is the time complexity of the below function?

```
int fun(int n)
{
    int count = 0;
    for (int i = n; i > 0; i /= 2)
        for (int j = 0; j < i; j++)
            count += 1;
    return count;
}
```

Solution:

For an input integer n, the innermost statement of fun() is executed following no. of times.

$$n + n/2 + n/4 + \dots 1$$

So time complexity T(n) can be written as

$$T(n) = O(n + n/2 + n/4 + \dots 1) = O(n)$$

Q.9) Consider the following C function.

```
int fun1 (int n)
{
    int i, j, k, p, q = 0;
    for (i = 1; i < n; ++i)
    {
        p = 0;
        for (j = n; j > 1; j = j/2)
            ++p;
        for (k = 1; k < p; k = k*2)
            ++q;
    }
    return q;
}
```

}

Which one of the following most closely approximates the return value of the function fun1?

(A) n^3

(B) $n (\log n)^2$

(C) $n \log n$

(D) $n \log(\log n)$

Solution: (D)

Explanation: The outer for loop runs $O(n)$ times. Among the two inner for loops, the first one runs $O(\log n)$ times, so $p = O(\log n)$. The second inner for loop runs $O(\log p)$ times i.e., $O(\log \log n)$ times. So q will be $O(\log \log n)$ at the end of one iteration, and $n * O(\log \log n)$ or $O(n \log \log n)$ at the end of n iterations. So the answer is D.

Q.10) What is the time complexity of the following function fun()? Assume that $\log(x)$ returns $\log$ value in base 2.

```
void fun()
{
    int i, j;
    for (i=1; i<=n; i++)
        for (j=1; j<=log(i); j++)
```



```
    printf("hello world");  
}
```

Solution: $O(\log n!)$

Explanation: The time complexity of the above function can be written as: $O(\log 1) + O(\log 2) + O(\log 3) + \dots + O(\log n)$ which is nothing but $O(\log n!)$